

Abhinav Yadav

Email: me@abhiyadav.in Mobile: +91 8630087242 Address: Noida, 201301, India

<https://abhiyadav.in> <https://github.com/abhinav-yadav-official> <https://linkedin.com/in/abhiyd>

OVERVIEW

Full Stack SDE III / Platform Engineer at *Instahyre*, 6.5+ years across backend, platform, and search. Systems serve 2M+ users, 50M+ monthly requests, 10M+ indexed docs, and 2B+ row tables. Owns 2 internal platforms (*Forge* deployments, *ESMultiWrite* search migrations) and the *P0/P1* review gate spanning 4 teams and 15+ engineers. Drove 42% drop in *P0/P1* incident frequency and a 4.8s to 85ms p95 cut on the busiest candidate API.

TECHNICAL SKILLS

Languages: Python, Go, Java, JavaScript, TypeScript, Painless (Elasticsearch)
Backend: Django, FastAPI, Flask, Node.js, Tastypie
Databases: MySQL, PostgreSQL, Redis (cache and queue)
Search: Elasticsearch, OpenSearch, Painless scripting, BM25, custom scoring
Async / Messaging: Celery, RabbitMQ, task queues, scheduled jobs
Frontend: React, Next.js, AngularJS, HTMX
Cloud / Infra: AWS (EC2, RDS, S3, Lambda, CloudFront, Secrets Manager), Docker, Nginx, Linux
Observability: Datadog (APM, dashboards), CloudWatch, Sentry, structured logging
System Design: distributed systems, online schema migrations, search index migrations, caching, RBAC

WORK EXPERIENCE

Software Development Engineer III, *Instahyre*, Platform Architecture, Noida, India Apr 2024 - Present

- Designed and shipped *Forge*, *Instahyre*'s internal deployment platform and now the only deploy path for the backend org. Replaced manual SSH deploys with guarded auto-merge and one-command deploy/rollback for 15+ engineers and 200+ weekly deployments. Deploy time fell from 30 min to 4 min, rollback *MTTR* from 20+ min to under 90 sec, and a recurring class of deploy-related *P0* incidents was eliminated.
- Replaced a proposed caching layer for *Instamatch*, the busiest candidate ranking API (8M+ requests/day), with a single *OpenSearch* query and scoring logic pushed into *Painless*. p95 latency dropped from 4.8s to 85ms, app-server load fell ~60%, and the system absorbed 2x concurrent-user growth without added capacity.
- Led the zero-downtime *BIGINT* migration of the opportunity table (2B+ rows, critical write path) past its 32-bit primary-key ceiling. Used dual-write, online backfill, and atomic cutover; customer impact stayed under one second. Caught bulk-insert deadlocks producing ID gaps that would have corrupted referential integrity. This became the template for later large-table migrations.
- Built *ESMultiWrite*, the search index migration system used across 10M+ documents and 12+ production indices. It made alias swaps, parallel rebuilds, cross-version readers, and rollback paths explicit instead of one-off scripts per migration, so teams could rebuild an index, test the new reader, and switch traffic through aliases with a known fallback path. Used it for the ES 7 to *OpenSearch* 3 cutover with zero search downtime; new index work now follows the same migration contract by default.
- Led the Python 2 to Python 3 migration of a 150K+-line legacy codebase. Identified compatibility risks, sequenced 6 months of delivery without blocking feature work, and cut over with zero production regressions.
- Cut *P0/P1* incident frequency 42% over 18 months across the backend org (15+ engineers, 4 teams) by turning incident review into an engineering gate. Built the taxonomy, runbooks, and review workflow, then mapped recurring failure modes back to schema rules, index-change policy, and release checks. The process made risky changes visible before deploy, protected SLAs for 2M+ users, and gave teams a concrete checklist before production changes.
- Acted as sole sign-off for schema and search-index changes across the backend org. Wrote review standards for alias versioning, cache invalidation ownership, schema evolution gates, service boundaries, and rollback criteria. The review covered hot tables, high-traffic indices, and changes touching ranking or deploy safety. Approved 100+ index and migration PRs over 18 months with zero rollbacks; 2 teams that joined later adopted the same standards without changes.
- Wrote migration and async-debugging playbooks used by 3 teams to ship schema and index changes without my involvement, including 2 multi-million-row migrations using the *BIGINT* cutover pattern. Mentored 30+ engineers across system design, race-condition debugging, and postmortems.

WORK EXPERIENCE

Senior Software Engineer, *Instahyre, Scale, Reliability & Modernization, Remote*

May 2022 - Mar 2024

- Decoupled a monolithic production runtime into three tiers: web serving, dedicated Celery worker pools, and the outbound processing pipeline. Each tier could scale and release on its own cadence. Web-tier *p99* tail latency dropped 35% because outbound bursts no longer competed with user-facing traffic, and the outbound team shipped roughly 3x faster once they were off the shared deploy.
- Designed a reusable Redis cache-aside layer with per-key TTLs, event-driven invalidation, and pooled connections. Cut *p95* latency 38% on high-traffic endpoints. Adopted across 20+ endpoints company-wide.
- Stabilized production load. Resolved MySQL replica lag, hot-table deadlocks, CPU saturation, lock contention, and Celery retry storms across tables with 100M+ rows and queues running 500K+ tasks per day.
- Extended search and ranking with Java services and Painless scripts for custom scoring and online backfills. Ranking parameters could be tuned in production without an application release, letting the product team run scoring experiments on real traffic.
- Built tooling for the migrations the team kept doing by hand: a Node.js PDF-to-HTML conversion pipeline, AngularJS bulk refactor scripts, and AWS Lambda data-fix utilities. Migration prep time fell from days to hours and the same tools were reused across 6+ later projects.
- Hardened production access after a near-miss: least-privilege SSH, Unix user isolation, private network routing, and AWS Secrets Manager with automated credential rotation. Passed the next external security audit with zero access-control findings.

Software Engineer, *Instahyre, Product Systems & Security, New Delhi, India*

Aug 2019 - Apr 2022

- Shipped *Team Management* end-to-end (adopted by 1,000+ employer companies, 8,000+ recruiter seats): product workflow, RBAC authorization, audit trails, and impersonation safeguards. Core feature behind the premium pricing tier.
- Closed authorization bypasses across backend APIs and frontend flows that exposed premium features, tightening access control across the full request path.
- Eliminated production race conditions, non-atomic state transitions, and MySQL full-table scans via Django and Celery workflow fixes, composite and covering indexes, and EXPLAIN-driven rewrites.
- Cut mean incident detection time from ~18 min to under 9 min by rolling out monitoring and tracking across API and background processing tiers: structured alerts, distributed tracing, and error-rate aggregation dashboards.
- Refactored legacy AngularJS flows and supporting JavaScript/Node.js scripts: pulled duplicated UI logic into shared modules, automated data cleanup that had been manual, and made high-risk product changes reviewable in a single diff instead of 4-5.

Software Engineer Intern, *Fosterphi India, Greater Noida, India*

Jul 2018 - Aug 2018

- Handwritten digit recognition web app: Flask, Keras/TensorFlow, MNIST, JavaScript canvas frontend.

OSS PROJECTS

LeetDrill, *LeetCode spaced-repetition system*

- Built review scheduling around the SM-2 algorithm, recall scoring, and problem history.
- Captures submissions through a Chromium extension hooked into LeetCode's submit flow.
- Backend in Go and PostgreSQL; review UI in HTMX.
- Tracks weak topics over time so review sessions surface older missed problems before newer solved ones.

Ichnos, *Crawler and search engine*

- Built URL scheduling, crawl politeness, HTML cleanup, and on-disk inverted-index construction.
- Implemented BM25 ranking with tunable parameters to understand search mechanics from first principles.

Kleos, *Cold-outreach workflow engine*

- Handles sequences, throttling, suppression lists, reply detection, and async worker execution.
- Uses task-queue retries and per-recipient state so one stalled send does not block the sequence.

EDUCATION

Dr. A.P.J. Abdul Kalam Technical University (AKTU), Lucknow, India (B.Tech, Computer Science)

2015 - 2019